Seminar 2

CUDA Accelerated NMS V1 · V2 · V3 with Numba

Greedy hard-NMS, batched bitmasks, and Matrix NMS on Tesla T4

Team: Tuấn · Tân
Course: Applied Parallel Programming

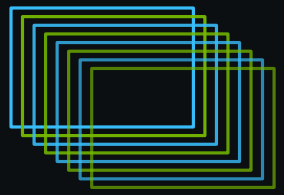
CPU → V1 full IoU → V2 packed mask → V3 Matrix NMS → T4 evidence

What We Actually Implemented

From a greedy CPU reference to three CUDA designs

Problem Motivation

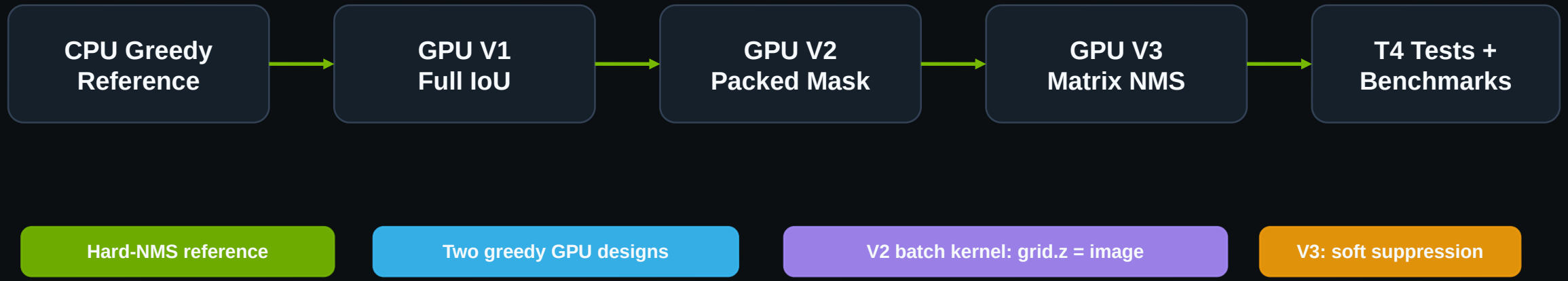
Object detectors produce many overlapping boxes



NMS removes redundant detections while keeping the highest-confidence box.

Project Scope: Three CUDA NMS Designs

Track A4 · real-time object-detection post-processing



Implemented: CPU baseline + GPU V1 + GPU V2 + GPU V3
V1/V2 checked against greedy CPU/torchvision; V3 checked against Matrix-NMS CPU oracle
Evidence: 50/50 CUDA tests plus repeated Tesla T4 benchmarks

CPU Greedy NMS

Baseline algorithm and complexity

Sorting: O(N log N)

Greedy suppression: ~ O(N^2)

IoU repeated many times

CPU Baseline Architecture

End-to-end reference implementation

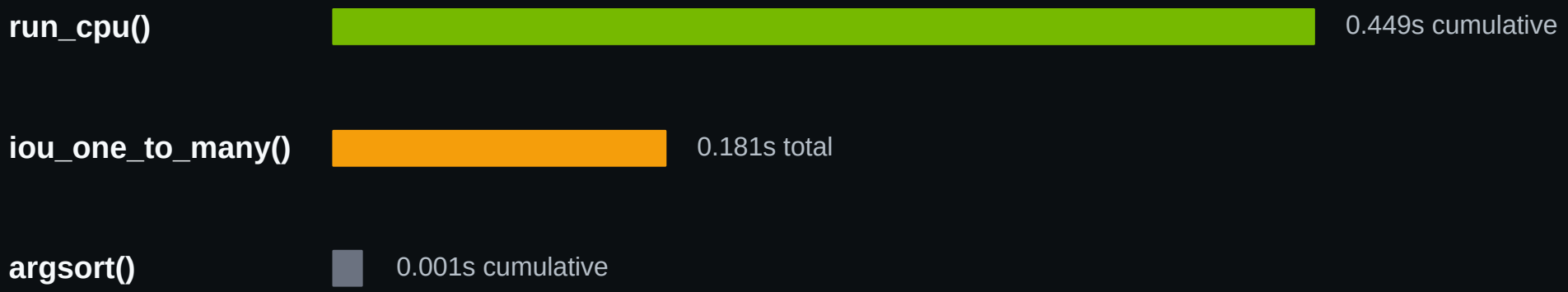
WHY THIS MATTERS

CPU baseline is the greedy hard-NMS reference.
Profiling identifies the pairwise IoU bottleneck.
V1/V2 match this output; V3 uses a Matrix-NMS oracle.

CPU Profiling: Pairwise IoU Is the Target

Local CPU · N = 10,000 · cProfile evidence

18,313	0.449s	6,100	40.3%	0.001s
function calls	total runtime	IoU calls	IoU share	argsort



Three GPU Designs, Three Trade-offs

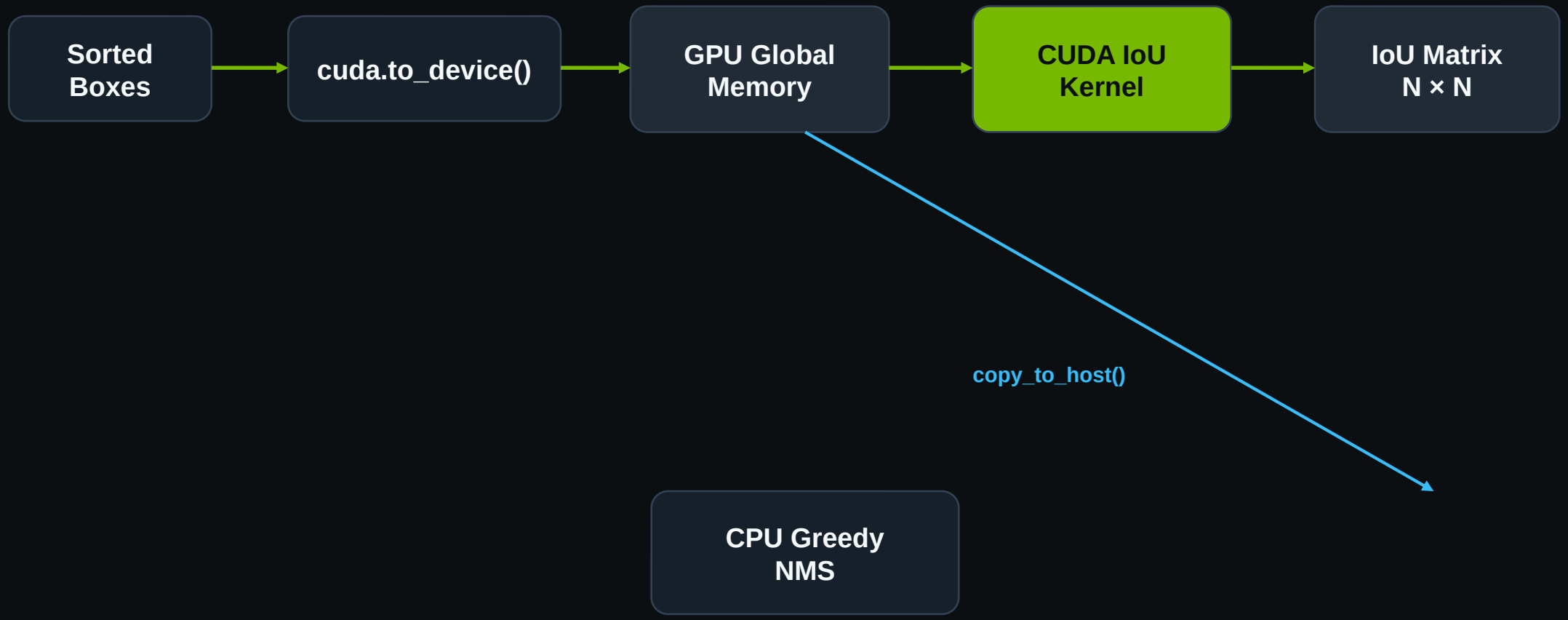
Each version removes a different bottleneck

V1: simple, exact greedy result, expensive transfer

V2: exact greedy + batch kernel · V3: parallel soft suppression

GPU Version 1 Architecture

Parallel IoU matrix, CPU greedy suppression



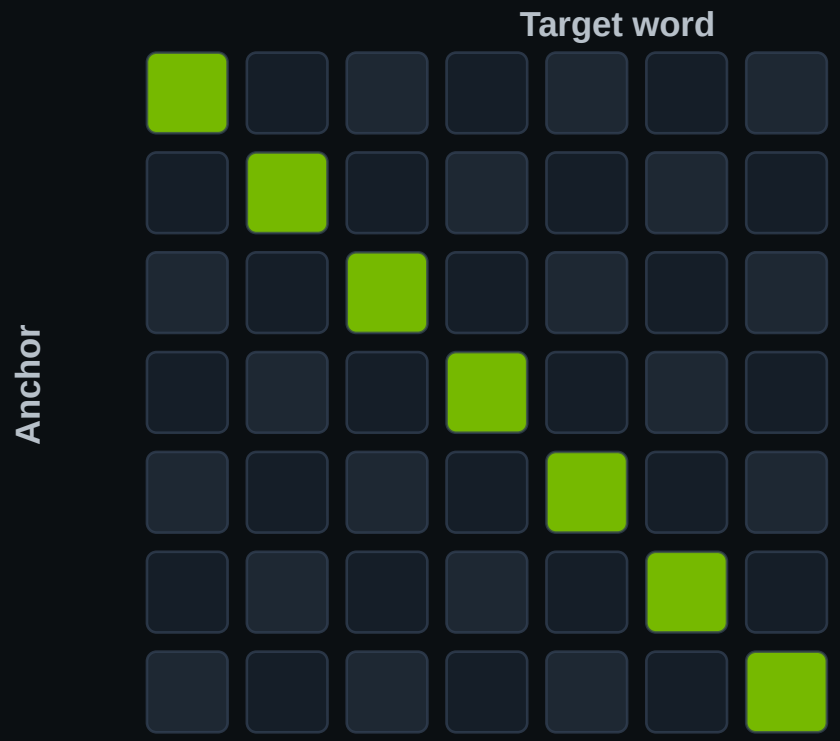
GPU does the $O(N^2)$ pairwise work

CPU keeps simple greedy control flow

V1 is easy to test and explain

GPU V2: Batched Packed Suppression Mask

grid = (anchor word, target word, image)



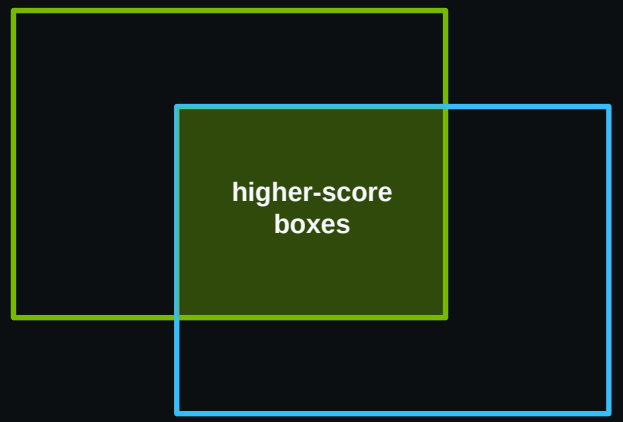
Thread(anchor)
→
one uint64 mask word

blockIdx.x = anchor · y = target · z = image

SoA layout enables coalesced coordinate reads.
Each block caches 64 target boxes in shared memory.
One bit records whether IoU exceeds the threshold.

GPU V3: Matrix NMS Soft Suppression

Two CUDA kernels remove the greedy dependency



Kernel 1: max IoU against higher-score boxes

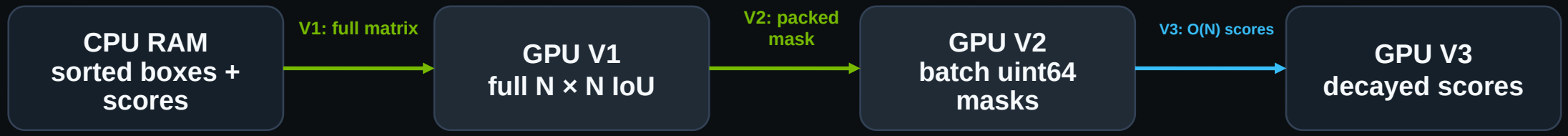
Kernel 2: parallel score decay — linear or Gaussian

Host: copy final scores and apply score threshold

V3 returns Matrix-NMS results, not greedy hard-NMS indices.
Correctness is checked against matrix_nms_reference().
The 4.092 ms result is an algorithmic trade-off.

What Each Version Transfers

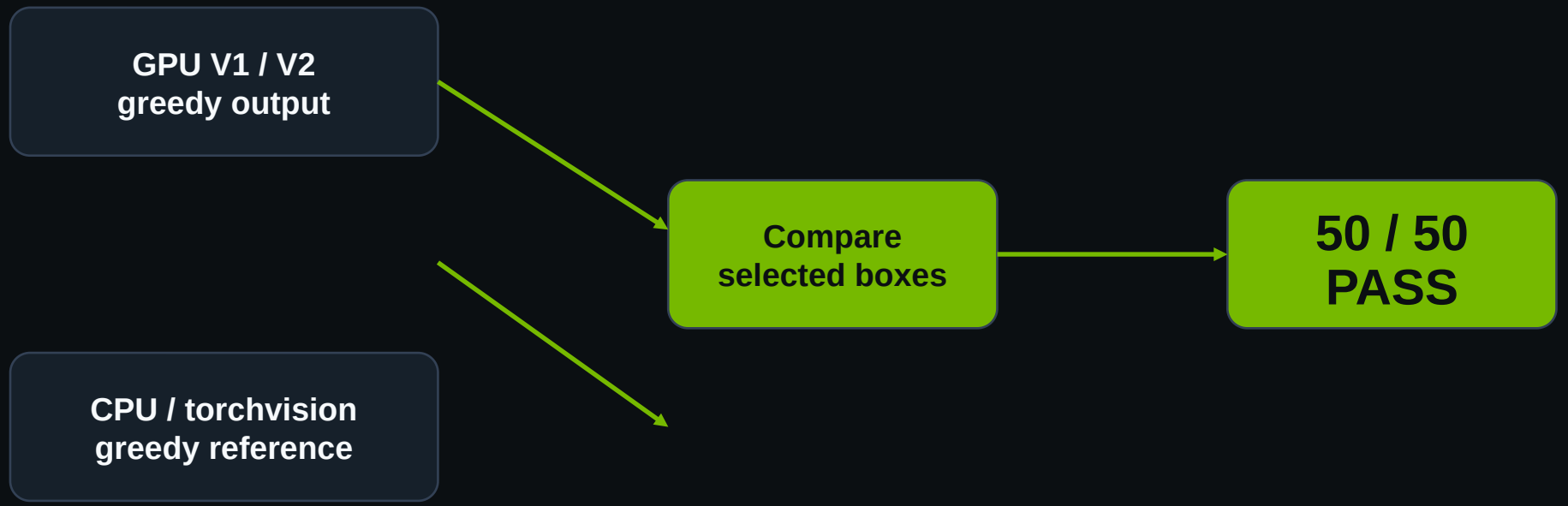
Separate designs with progressively smaller returned data



Separate designs: V1 $O(N^2)$ floats · V2 $O(B \cdot N^2/64)$ masks · V3 $O(N)$ scores, different algorithm

Correctness & Environment

Verified on Tesla T4 before reporting performance



Tesla T4 · compute capability 7.5

Python 3.12.13 · NumPy 2.0.2

V3 ↔ Matrix-NMS CPU oracle · not greedy NMS

Verified Tesla T4 Benchmark Method

Repeatable end-to-end latency after CUDA warmup

N ∈ {100, 1,000, 10,000} boxes

Single-image + V2 batch 32

Tesla T4 · compute capability 7.5

Single-Image Latency on Tesla T4

Median end-to-end latency (ms) · 7 repeats after 2 warmups

Latency (ms)

N	CPU	V1	V2	V3
100	2.389	0.777	2.309	1.395
1,000	38.777	7.454	3.729	1.376
10,000	1125.272	226.107	31.599	4.092

At N = 10,000

V1: 5.0× CPU
V2: 35.6× CPU

V3 = Matrix NMS / soft suppression
Speed is a trade-off only — output is not equivalent to greedy hard NMS.

V2 Batch-Size 32: Correct, Target Missed

Latency input: 32 images × 10,000 boxes · end to end

VERIFIED RESULT

Correctness: PASS at B=32, N=50
Latency input: B=32, N=10,000
Median: 1002.239 ms / batch
Median: 31.320 ms / image
Stddev: 214.264 ms

WHY IT MISSES <5 MS / BATCH

$O(B \cdot N^2)$ pairwise work

Packed mask copied to host · greedy resolution on CPU

Performance Trade-offs by Design

The fastest result does not always preserve the same NMS semantics

GPU V1
full IoU matrix
CPU greedy

GPU V2
packed masks
CPU greedy

GPU V3
Matrix NMS
soft suppression

Small N: launch overhead matters

V2 wins hard-NMS single-image

V3 is fastest, but semantics differ

Limitations / Honest Conclusion

Measured gains are real; algorithmic and batch limits remain

CURRENT LIMITATIONS

V1 transfers the full IoU matrix GPU → CPU.
V2 packs uint64 masks, but final greedy decisions stay on CPU.
V3 removes greedy dependence with Matrix NMS, but its output is not equivalent to hard NMS.

WHAT THE RESULTS SUPPORT

V2 meets the ≥15× single-image target

Batch-32 latency target remains unmet

Final Status & Contributions

Seminar 2 final · verified T4 evidence

- ✓ 50 / 50 CUDA tests passed
- ! Batch 32: 1002.239 ms — target missed
- ✓ V1: 5.0× CPU at N=10,000
- ✓ Tuấn — CPU baseline, tests, repository
- ✓ V2: 35.6× CPU at N=10,000
- ✓ Tân — GPU V1 / V2 / V3, benchmarks

V2 meets ≥15× for single image; <5 ms / batch is not achieved

Q&A: V2, V3, and Batch Semantics

Short answers grounded in the code and T4 evidence

Why does V2 miss <5 ms / batch?	$O(B \cdot N^2)$ work, mask download, and CPU greedy resolution.
Does V3 match torchvision NMS?	No. V3 matches the Matrix-NMS CPU oracle.
What does batch size 32 mean?	32 independent images; grid.z selects the image.
Why use Numba CUDA?	Custom kernels while keeping the Python project pipeline.

Appendix: V2 Hard NMS vs V3 Matrix NMS

Why the outputs are validated against different references

GPU V2 — Greedy Hard NMS

stable sort on host
GPU builds batched uint64 masks
copy packed masks to host
CPU resolves greedy bitwise OR

matches CPU / torchvision indices

GPU V3 — Matrix NMS

stable sort on host
GPU finds max IoU per box
GPU applies linear / Gaussian decay
copy scores and threshold

matches Matrix-NMS CPU oracle

References & Reproducibility

Code, correctness evidence, and measured results

Applied Parallel Programming — Track A4: Real-Time Non-Maximum Suppression.
Numba CUDA kernels: src/gpu_v1.py, src/gpu_v2.py, src/gpu_v3.py.
Greedy reference: CPU baseline and torchvision.ops.nms tests.
Matrix-NMS reference: matrix_nms_reference() and CUDA parity tests.
Tesla T4 evidence: pytest_t4_final.txt, benchmark_t4_single.json, batch32_t4.json.

Thank you